INFORMATION TECHNOLOGIES — Final Exam — SOLUTIONS

Author: **Víctor Barceló** **Course:** Third — Information Technologies **Academic year:** 2025/2026 **Exam duration:** 2 hours **Round:** Practice test.

Case Study: Real Zaragoza — Saving the Club Through Data

Real Zaragoza, S.A.D. is one of the most historically significant clubs in Spanish football. Founded in 1932 and based in Zaragoza, Aragon, the club has won six Copa del Rey titles, the 1964 Inter-Cities Fairs Cup, and the 1995 UEFA Cup Winners' Cup. However, after being relegated from La Liga in 2013, the club has spent over a decade in the Segunda Division — Spain's second tier — and is now facing the very real risk of dropping to the third division. Finishing 18th out of 22 teams in the 2024-25 season, Real Zaragoza is under enormous pressure.

The club's board has approved a strategic digital transformation initiative. The goal is to build a comprehensive data platform that improves player performance, sharpens tactical decisions, and enables data-driven operations that will help the club survive in the current category and eventually compete for promotion back to La Liga. The platform must be built from the ground up, covering both operational systems and a full analytical infrastructure, including a Data Warehouse.

The platform must handle several layers of information with very different characteristics:

- **Operational data**: player contracts, medical records, injury logs, fitness test results, match schedules, ticketing, and administrative workflows. This data must be immediately consistent and must support daily operations.
- **Performance data**: detailed statistics collected during training sessions and matches — such as distance covered, sprint count, pass accuracy, shots on target, expected goals (xG), physical load index, and injury recurrence rates — tracked per player, per match, and per season. This data is generated in near real time during matches and training by wearable sensors worn by all 25 players.
- **Tactical and scouting intelligence**: opponent analysis reports, set-piece data, formation breakdowns, video clip tags, and recruitment assessments on transfer targets. This data is semi-structured and its schema may evolve as scouting methodologies change.
- **Fan and commercial data**: ticket sales, season pass subscriptions, merchandise revenue, and fan demographics for the 20,000-seat Ibercaja Stadium, which opened in 2025.
- **Social and media monitoring**: sentiment data scraped from social media, press coverage summaries, and fan feedback surveys.

The technical team has identified that data is highly heterogeneous: some is structured (match results, contracts), some is semi-structured (scouting reports, tactical notes), and some is unstructured (video footage, social posts). Furthermore, certain data must be available in real time (live match tracking during a game), while

other data is better suited for historical, long-term analysis (multi-season performance trends).

The final objective is not only to store data, but to build **end-to-end data pipelines** that move data from source systems into an analytical layer, so that analysts, coaches, and the management team can answer questions such as:

- "Which players maintain high performance under high-pressure, away matches?"
- "What tactical formations yield the best results against top-of-the-table rivals?"
- "What is the correlation between training load and injury occurrence across a full season?"

Question 1 — Database Selection and Architecture (3 points)

The technical team must decide how to architect the database layer of the platform.

Part A (1.5 points)

Evaluate whether the team should implement the system using:

- A **relational database (SQL)**,
- A **non-relational database (NoSQL)**, or
- A **hybrid approach**, where different types of data are stored in different database technologies.

List and describe **at least 6 reasons** (pros or cons) that justify the recommended approach. Each reason must be clearly related to the Real Zaragoza platform described in the case.

SOLUTION — Part A

The recommended approach is a **hybrid architecture**. The data in this platform is too heterogeneous in structure, consistency requirements, and access patterns to be managed well by a single technology.

1. **Relational databases enforce integrity for critical operational data.** Player contracts, match schedules, ticketing, and medical records have well-defined schemas and require strict ACID guarantees. PostgreSQL prevents inconsistencies such as a player appearing in a lineup without a valid contract, or a ticket being sold for a sold-out match.
2. **Scouting and tactical data has a flexible, evolving schema.** Scouting reports and formation breakdowns change structure as methodologies evolve — new metrics are introduced each season (e.g., PPDA, xA, progressive carries). A rigid relational schema would require costly migrations every time. A document-oriented NoSQL database handles this naturally.
3. **High-frequency sensor data requires write throughput that strains relational databases.** Wearable sensors on 25 players generate a continuous event

stream during training and matches. Relational databases are not optimized for this type of high-volume append workload; a wide-column store or time-series database can ingest it far more efficiently.

4. **Unstructured social and media data does not fit tabular storage.** Sentiment data, press summaries, and fan feedback surveys are unstructured or semi-structured. Forcing them into a relational schema would require artificial normalization or text blobs, losing queryability. A document store preserves their natural structure.
5. **A hybrid approach uses the right tool for each concern.** Separating the OLTP layer (relational), the document layer (NoSQL), and the analytical layer (Data Warehouse) follows the principle of purpose-fit storage. Each system is optimized for its workload, improving both performance and maintainability.
6. **Relational databases are the natural fit for the Data Warehouse dimensional model.** The analytical layer — fact and dimension tables — is inherently relational. Joins between fact and dimension tables, aggregation queries, and window functions are all first-class operations in SQL, making a relational engine the correct foundation for the OLAP layer.
7. **Fan and commercial data benefits from the transactional guarantees of SQL.** Ticket sales and season pass subscriptions involve financial transactions. Partial sales, double bookings, or race conditions during high-demand events (match day spikes at the 20,000-seat Ibercaja Stadium) must be prevented by the database engine.

Part B (1.5 points)

Assume the team has decided to incorporate a NoSQL component into the architecture to handle the performance and scouting data. Among the NoSQL technologies studied in class, identify the **most suitable one** for this component and provide **at least 6 reasons** that justify your choice, relating each reason to the specific requirements of the case.

SOLUTION — Part B

The most suitable NoSQL technology is **MongoDB**.

1. **Flexible document model for player performance data.** A goalkeeper's performance metrics (saves, goals conceded, distribution accuracy) differ entirely from a striker's (shots on target, xG, pressing actions). MongoDB's schema-less documents store any combination of fields without a fixed structure, accommodating every player position naturally.
2. **Schema evolution without migrations.** As new performance metrics are adopted (e.g., expected assists, defensive line breaks), MongoDB allows adding new fields to documents without altering existing records. This avoids the costly

ALTER TABLE operations that a relational schema would require and aligns with the case's note that "its schema may evolve."

3. **Embedded documents reduce join complexity for performance queries.** A single MongoDB document can embed all of a player's metrics for one match, eliminating the need for multi-table joins when the coaching staff asks: "Show me every metric for Francho Serrano in the last 5 matches." In a relational model, this would require joining several normalized tables.
4. **Rich aggregation pipeline for analytical queries.** MongoDB's aggregation pipeline supports filtering, grouping, projection, and sorting directly on the performance collection. This allows analysts to run ad-hoc queries — such as "average sprint count per position in away matches" — without moving data to a separate system.
5. **Well-suited for scouting report documents.** Recruitment assessments on transfer targets are naturally document-like: a mix of structured ratings, free-text tactical notes, video clip references, and match statistics. MongoDB stores all of this in a single, queryable document.
6. **Secondary indexing for common access patterns.** MongoDB supports compound indexes (e.g., on `player_id + match_date`), text indexes for searching scouting notes, and wildcard indexes for flexible fields. This allows performance queries to be served efficiently even as the data volume grows.
7. **Comparison with Cassandra.** Cassandra would handle high write throughput efficiently, but its rigid partition key design requires query patterns to be known at schema design time. For scouting data where queries vary widely and evolve with tactical needs, MongoDB's flexible querying is a better fit.
8. **Comparison with Redis.** Redis is an in-memory data structure store, suited for caching and real-time counters. It is not appropriate for persisting multi-season player performance histories and scouting reports at scale.

Question 2 — Dimensional Modelling (3 points)

The analytics team wants to build a Data Warehouse to support long-term performance analysis. The most critical analytical use case is understanding **player performance per match**, so that coaches and analysts can identify trends across seasons, competitions, and match conditions.

Part A — Design the Dimensional Model (1.5 points)

Design a **dimensional model** (star schema or snowflake schema) for the following analytical question:

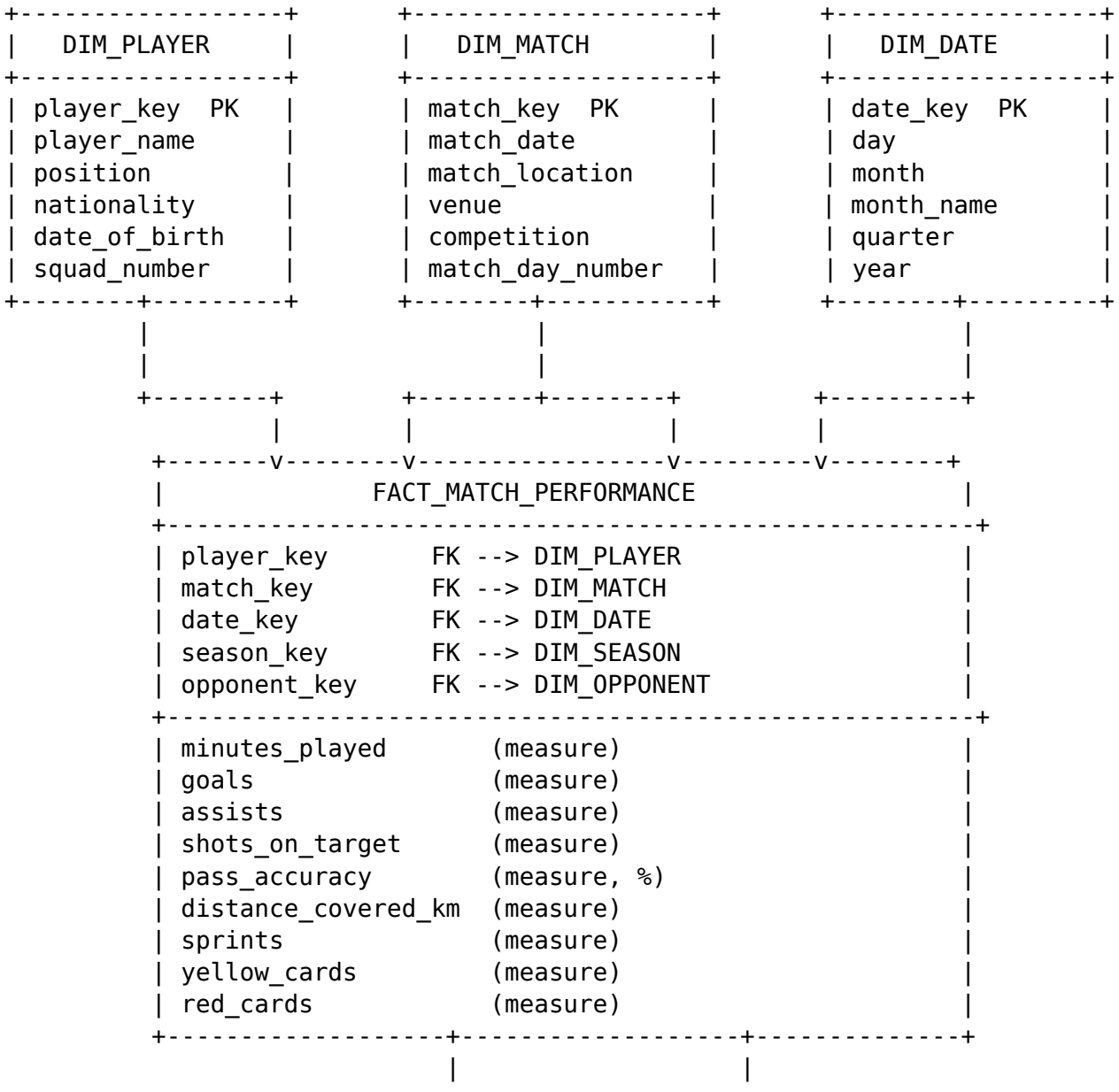
> "How has each player performed, match by match, throughout each season and competition? How does performance vary depending on whether the match is home or away, and against which type of opponent?"

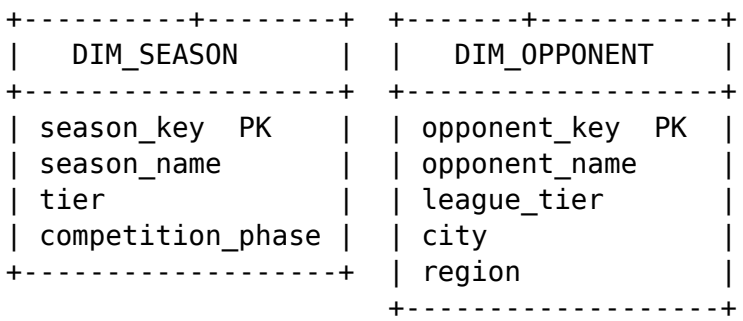
Your model must meet the following requirements:

- At least **1 fact table** with a minimum of **6 quantitative measures** relevant to player performance.
- At least **4 dimension tables**, each with meaningful attributes.
- Primary keys and foreign keys must be clearly identified.
- **Draw the diagram** using box-and-line notation. Label all tables, keys, and the most relevant attributes of each table.

SOLUTION — Part A

The recommended design is a **star schema**. Five tables surround the central fact table.





Part B — Analyse the Model (1.5 points)

Based on the dimensional model you designed in Part A, answer the following questions:

1. A senior analyst wants to calculate the **total distance covered by each player per season**, filtering only for home matches. Identify which tables and joins are needed to answer this query from your dimensional model. No need to write exact SQL syntax — describe the query logic in plain language.
2. The coaching staff asks: "What is the average pass accuracy of our midfielders when playing away matches in the second half of the season (from January onwards)?" Identify which dimensions and measures are needed and explain how you would filter and aggregate the data using your model.
3. Explain the difference between a **star schema** and a **snowflake schema**. In the context of the Real Zaragoza Data Warehouse, under what circumstances would you choose one over the other? Justify your answer.

SOLUTION — Part B

Question 1. The query involves three tables: - `FACT_MATCH_PERFORMANCE` — to read the `distance_covered_km` measure. - `DIM_PLAYER` — joined on `player_key` to retrieve `player_name` for grouping. - `DIM_MATCH` — joined on `match_key` to filter rows where `match_location = 'home'`. - `DIM_SEASON` — joined on `season_key` to group results by `season_name`.

Logic: join the four tables, filter to home matches only, then sum `distance_covered_km` grouped by player and season.

Question 2. The query involves: - `DIM_PLAYER` — filter by `position = 'Midfielder'`. - `DIM_MATCH` — filter by `match_location = 'away'`. - `DIM_DATE` — filter by `month >= 1` (January onwards in calendar terms; if the season starts in August the "second half" begins around January, so `month IN (1, 2, 3, 4, 5, 6)` is the correct filter). - `FACT_MATCH_PERFORMANCE` — aggregate the `pass_accuracy` measure using AVG.

Logic: join all four tables, apply the three filters, then compute the average of `pass_accuracy`. The result can be grouped by `player_name` to see individual midfielders, or computed as a single value for all midfielders.

Question 3. A **star schema** keeps all dimension attributes in a single, denormalized table. For example, DIM_OPPONENT contains `opponent_name`, `league_tier`, `city`, and `region` all in one flat table. Queries require fewer joins, are simpler to write, and typically execute faster. The trade-off is some data redundancy (e.g., the city name is repeated for every match against the same opponent).

A **snowflake schema** normalizes the dimension tables further. DIM_OPPONENT could be split into DIM_OPPONENT, DIM_CITY, and DIM_LEAGUE — each linked by foreign keys. This reduces redundancy and storage, but requires more joins per query and is harder to read and maintain.

For Real Zaragoza's Data Warehouse, the **star schema is the better choice**. The dataset is not large enough for storage savings to matter significantly, and coaches and analysts need to run ad-hoc queries quickly and without deep SQL expertise. The star schema's simplicity reduces the risk of query errors and improves performance for the aggregation-heavy workloads typical of performance analysis.

Question 3 — Data Pipelines and Analytical Architecture (2.5 points)

The technical team now needs to design the full data architecture, connecting operational sources to the analytical layer.

Part A (0.75 points)

Distinguish between an **operational (OLTP)** information system and an **analytical (OLAP)** information system in the context of Real Zaragoza's platform. Provide one concrete example of each, taken from the case description. Which of the two types of systems would be most appropriate to support the goal of analyzing multi-season performance trends and making promotion decisions? Justify your answer.

SOLUTION — Part A

An **OLTP (Online Transaction Processing)** system is designed to support day-to-day operations. It processes short, fast transactions — inserts, updates, and deletes on individual records — and prioritizes data consistency and availability. From the case: the **ticketing system** is an OLTP system. When a fan purchases a ticket for a match, the system must record the transaction atomically, update seat availability instantly, and prevent double bookings, all in milliseconds.

An **OLAP (Online Analytical Processing)** system is designed for complex read queries over large volumes of historical data. It supports aggregations, multi-dimensional analysis, and trend detection rather than individual record updates. From the case: the **Data Warehouse storing multi-season player performance statistics** is an OLAP system. It enables queries such as "what is the correlation between physical load and injury occurrence across the last three seasons?"

For the goal of analyzing multi-season performance trends and making promotion decisions, **OLAP is the appropriate system**. The questions involved (trend detection,

cross-season comparisons, aggregate KPIs) are inherently analytical and historical — they require scanning millions of records and computing aggregations, not fast row-level operations. An OLTP system is not designed for this kind of query and would perform poorly under the load.

Part B (0.75 points)

To populate the Data Warehouse, the team must design a **data pipeline** using an ETL (Extract, Transform, Load) process. Describe the three main stages and explain what each stage would involve for **at least two of Real Zaragoza's data sources** described in the case. Mention at least one concrete challenge for each stage.

SOLUTION — Part B

Extract The first stage reads raw data from the source systems without modifying it. - For the **operational database (player medical and fitness records)**: data is extracted using SQL queries or Change Data Capture (CDC) to capture only records that have changed since the last ETL run. Challenge: the operational database is live during extraction; if a player's injury record is updated mid-extraction, the snapshot may be inconsistent. Snapshot isolation or incremental extraction with timestamps mitigates this. - For **wearable sensor data (performance events)**: raw JSON or CSV files are exported from the sensor platform after each session. Challenge: sensors may disconnect or lose signal, producing gaps or out-of-order events in the data. These must be detected during extraction, not silently passed downstream.

Transform The second stage cleans, validates, and reshapes the data to conform to the Data Warehouse schema. - For **player records**: standardize player identifiers (the operational DB and sensor platform may use different IDs for the same player), validate metric ranges (pass accuracy cannot be negative or exceed 100%), derive new fields (physical load index computed from sprint count and distance), and handle null values for players who did not play in a given match. - For **match data**: map `match_location` values (e.g., "H", "home", "casa" from different sources) to a canonical "home/away" code; resolve opponent names to the correct `opponent_key` in DIM_OPPONENT. - Challenge across all sources: referential integrity — every foreign key in the fact table must resolve to an existing dimension record. If a new player signs mid-season, their `player_key` must be inserted into DIM_PLAYER before their performance rows are loaded into the fact table.

Load The third stage writes the transformed data into the Data Warehouse. - Use incremental loading (append new rows to the fact table) rather than full reloads, to avoid reprocessing five million existing rows on every pipeline run. - Challenge: ensuring idempotency — if the pipeline fails midway and is re-run, duplicate rows must not be inserted. This is typically handled by using upsert logic (INSERT … ON CONFLICT) or by staging data in a temporary table and merging only new records.

Part C — Graph Databases and Neo4j (0.5 points)

The analytics team proposes complementing the platform with a graph database (Neo4j) to analyze relational patterns between players, matches, and tactical events — for example, to detect passing networks or identify which player pairs perform best together.

1. Design a graph model for the following use case: "Find which pairs of players have the highest combined pass completion rate when playing together in the same match." Define at least **3 node types** and **2 relationship types**, including their most relevant properties.
2. Would Neo4j serve as a replacement for the relational operational database, for the Data Warehouse, or as a complementary system? Justify your answer by comparing Neo4j's strengths and limitations against at least two other database technologies studied in class.

SOLUTION — Part C

Question 1 — Graph model.

Node types: - **Player** — properties: `player_id`, `player_name`, `position`, `nationality` - **Match** — properties: `match_id`, `match_date`, `competition`, `match_location` - **Team** — properties: `team_id`, `team_name`, `league_tier`

Relationship types: - **(Player)-[:PLAYED_IN]->(Match)** — properties: `minutes_played`, `goals`, `assists` - **(Player)-[:PASSED_TO {match_id}]->(Player)** — properties: `total_passes`, `completed_passes`, `completion_rate`

To answer the use case query, traverse `PASSED_TO` relationships between all pairs of players who share a `PLAYED_IN` relationship to the same `Match` node, then rank pairs by their combined `completion_rate`. In Cypher this would involve matching two players connected by a `PASSED_TO` edge within a specific match context and computing the average completion rate for each pair across all matches.

Question 2 — Neo4j as complementary system.

Neo4j should serve as a **complementary system**, not as a replacement for either the relational OLTP database or the Data Warehouse.

- **vs PostgreSQL (relational OLTP)**: PostgreSQL provides ACID transactions, referential integrity, and structured schema enforcement — none of which Neo4j prioritizes. Player contracts, ticketing transactions, and medical records require consistency guarantees that a graph database is not designed to enforce. Neo4j cannot replace the operational layer.
- **vs the Data Warehouse (OLAP)**: The Data Warehouse is optimized for aggregation queries over millions of rows (SUM, AVG, GROUP BY) using columnar storage and partition pruning. Neo4j is optimized for relationship traversal — finding shortest paths, connected communities, n-hop neighbors. Computing "total goals per player per season" is trivially efficient in the DW and extremely inefficient in Neo4j. Neo4j cannot replace the analytical layer for aggregate KPIs.

- **As a complement**: Neo4j excels at queries that would require expensive recursive joins in SQL, such as detecting passing triangles, identifying player clusters by tactical partnerships, or tracing opponent vulnerability networks across multiple matches. These graph-native questions are the correct use case for Neo4j alongside the other systems in the architecture.

Part D — Query Optimization (0.5 points)

The `fact_match_performance` table holds 5 million rows spanning 15 seasons and is partitioned by `season_key`. A data analyst writes the following query to retrieve, for each player, their total goals and average pass accuracy in away matches during the 2025-26 season:

```
SELECT
    p.player_name,
    p.position,
    SUM(f.goals) OVER (PARTITION BY f.match_key) AS total_goals,
    AVG(f.pass_accuracy) OVER (PARTITION BY p.position ORDER BY m.match_date) AS avg_ac
    COUNT(*) AS appearances
FROM fact_match_performance f
JOIN dim_player p ON f.player_key = p.player_id
JOIN dim_match m ON f.match_key = m.match_id
JOIN dim_season s ON f.season_key = s.season_key
WHERE YEAR(m.match_date) = 2026
  AND m.match_location = 'away'
GROUP BY p.player_name, p.position;
```

1. The query contains multiple errors. Identify **all of them** and explain why each one is incorrect.
2. Write a corrected version of the query.
3. Suggest **two optimization strategies** (from those studied in class) that could further improve the performance of this query given the table's volume and partitioning scheme. Explain how each strategy helps.

SOLUTION — Part D

Question 1 — Errors.

Error 1 — Wrong foreign key column names in JOIN conditions. `f.player_key = p.player_id` is incorrect; the primary key in `DIM_PLAYER` is `player_key`, not `player_id`. `f.match_key = m.match_id` is incorrect; the primary key in `DIM_MATCH` is `match_key`, not `match_id`. These joins would fail (column not found) or silently produce a cross-join on engines that tolerate it.

Error 2 — Window functions combined with GROUP BY in the same query level. `SUM(f.goals) OVER (...)` and `AVG(f.pass_accuracy) OVER (...)` are window functions that operate on the unaggregated row set before any grouping. Placing them alongside a `GROUP BY` clause in the same SELECT causes a syntax or execution

error in most SQL engines, because the GROUP BY collapses rows first, leaving no row-level data for the window function to operate on.

Error 3 — Semantically incorrect PARTITION BY for total goals. `SUM(f.goals) OVER (PARTITION BY f.match_key)` computes the sum of goals within a single match (i.e., summing a single player's goals in one match — which is just that player's goals in that match). This does not produce a per-player total across all matches. To aggregate across all away matches per player, a simple `SUM(f.goals)` with `GROUP BY player_key` is the correct approach.

Error 4 — Function wrapper on date column prevents partition pruning and index use. `YEAR(m.match_date) = 2026` applies a scalar function to the indexed column `match_date`. Most database engines cannot use a B-tree index on `match_date` when it is wrapped in a function, forcing a full scan. More importantly, the fact table is partitioned by `season_key`, not by `match_date` — filtering on a derived year from a dimension join does nothing to trigger partition pruning. Filtering directly on `s.season_name = '2025-26'` (which resolves to a `season_key` value) allows the engine to skip 14 out of 15 partitions.

Question 2 — Corrected query.

```sql
SELECT
    p.player_key,
    p.player_name,
    p.position,
    SUM(f.goals)         AS total_goals,
    AVG(f.pass_accuracy) AS avg_accuracy,
    COUNT(*)             AS appearances
FROM fact_match_performance f
JOIN dim_player p ON f.player_key  = p.player_key
JOIN dim_match  m ON f.match_key   = m.match_key
JOIN dim_season s ON f.season_key  = s.season_key
WHERE s.season_name    = '2025-26'
  AND m.match_location = 'away'
GROUP BY p.player_key, p.player_name, p.position
ORDER BY p.player_name;
```

Note: `p.player_key` is added to the GROUP BY to guarantee uniqueness in case two players share the same name.

Question 3 — Optimization strategies.

Strategy 1 — Partition pruning via season_key filter. The table is partitioned by `season_key`. By filtering on `s.season_name = '2025-26'`, the query optimizer resolves this to a specific `season_key` value and reads only the partition for that season, discarding the other 14. This eliminates roughly 93% of the table from the scan, reducing I/O and execution time proportionally. The original query's `YEAR(m.match_date) = 2026` filter does not achieve this because it operates on a column from a joined dimension

table, not on the partition key.

Strategy 2 — Composite index on frequently filtered columns. Adding a composite index on `(season_key, match_location)` in the `FACT_MATCH_PERFORMANCE` table, and an index on `match_key` in `DIM_MATCH`, allows the database to quickly locate rows matching the filter `match_location = 'away'` within the already-pruned season partition, rather than scanning every row in it. Within the `DIM_PLAYER` table, an index on `player_key` (which is already the PK) ensures the join is resolved via index lookup rather than a hash join over the full dimension. Together, these indexes reduce the effective row set the query must process from tens of thousands to only the qualifying rows.

Question 4 — True or False: Comparative Analysis (1.5 points)

Select and complete **ONE** of the following options. Indicate whether the statement is **True or False**, and justify your answer by comparing the different database technologies studied in class.

OPTION A

> "Among the technologies studied in class, PostgreSQL is always the best option for building the Data Warehouse of a football club, because it guarantees data integrity through ACID transactions, and no other database technology studied in class can offer comparable consistency or analytical query support."

SOLUTION — Option A

FALSE.

PostgreSQL is a valid and cost-effective choice for a small-to-medium Data Warehouse, and its ACID guarantees do make it suitable for maintaining dimensional model integrity. However, the statement is false on two grounds.

First, "always the best option" is an overstatement. Dedicated columnar OLAP engines outperform PostgreSQL for analytical workloads at scale because they store data column by column, reading only the columns referenced in a query rather than full rows, and they apply aggressive compression that further reduces I/O. PostgreSQL's row-oriented storage is less efficient for the large aggregation queries that characterize a performance Data Warehouse.

Second, the claim that "no other technology can offer comparable consistency" is incorrect. MongoDB has supported multi-document ACID transactions since version 4.0. Cassandra offers tunable consistency, allowing operators to configure the level of consistency guarantees depending on the use case. Redis provides atomic operations on individual keys. None of these are identical to PostgreSQL's full relational transac-

tion model, but the blanket claim that no other technology can match its consistency is false.

For Real Zaragoza specifically, PostgreSQL is a reasonable choice given the club's scale and budget, but it is not categorically superior in all scenarios.

OPTION B

> "Among the technologies studied in class, Apache Cassandra is the most appropriate technology for storing Real Zaragoza's historical player performance data in a Data Warehouse, because its distributed architecture ensures linear scalability and its wide-column model is optimized for time-series and analytical read workloads."

SOLUTION — Option B

FALSE.

Cassandra's distributed architecture does provide linear horizontal scalability, and its wide-column model handles time-series write workloads efficiently. These properties make Cassandra a reasonable choice for ingesting high-frequency sensor events in real time. However, the claim that it is the most appropriate technology for a **Data Warehouse** is false.

A Data Warehouse must support complex, ad-hoc analytical queries: multi-table joins between fact and dimension tables, GROUP BY aggregations across multiple dimensions, window functions for running totals and rankings, and flexible filtering. Cassandra is fundamentally not designed for these operations. Its data model is query-driven — tables must be designed around specific, known query patterns, and it does not support joins between tables. Writing the query "give me the average pass accuracy per player per season filtered by match location and opponent tier" would be extremely difficult or impossible to express efficiently in Cassandra's query model (CQL).

Furthermore, Cassandra lacks a query optimizer, does not support window functions, and its consistency model (eventual consistency by default) is unsuitable for analytical results that must be accurate and reproducible. A relational engine or a proper OLAP system is far more appropriate for the Data Warehouse layer.

OPTION C

> "Among the technologies studied in class, MongoDB is the best choice for all of Real Zaragoza's data storage needs — from operational records to analytical reporting — because its flexible document model handles any type of data, and its aggregation pipeline can fully replace the need for a dedicated Data Warehouse and a dimensional model."

SOLUTION — Option C

FALSE.

MongoDB's flexible document model is well-suited for semi-structured data such as scouting reports and performance event documents, and its aggregation pipeline enables complex queries on document collections. However, the claim that it can handle **all** of the platform's needs and replace the Data Warehouse is false.

MongoDB's aggregation pipeline lacks the performance characteristics of a purpose-built OLAP engine. It does not use columnar storage, cannot apply partition pruning on a dimensional model, and does not support window functions for the kind of ranking and running-total analytics that coaches need (e.g., "running total of goals per player from match day 1 to match day 38"). Executing these queries over millions of performance documents in MongoDB would be significantly slower than the same queries on a properly indexed relational Data Warehouse.

Additionally, MongoDB does not enforce referential integrity. For operational data such as player contracts, ticketing, and medical records, the lack of foreign key constraints and transactional guarantees across multiple collections increases the risk of data inconsistencies — for example, a ticket sale referencing a match that no longer exists in the system.

A hybrid architecture — PostgreSQL for operational data, MongoDB for flexible semi-structured data, and a relational Data Warehouse for analytical workloads — is the correct approach.

End of exam — Solutions.